

Stratégie 1 — Buy & Hold : le backtest qu'on recalcule à la main

Pourquoi commencer par la « stratégie » la plus bête du monde ? Précisément parce qu'elle est bête : acheter une fois, ne plus rien faire. Son résultat est de l'arithmétique pure — quantité × prix, moins les frais. Si le backtest le plus simple ne retombe pas **exactement** sur ce qu'on calcule à la main, quelque chose est faux dans la chaîne données → fill → frais → équité, et toutes les stratégies « intelligentes » posées dessus hériteront du défaut. C'est notre **sol de vérité** — le plancher de confiance de la formation, qu'on rejouera côté vectorbt (section 02) puis en confrontation des deux moteurs (section 03). Buy & Hold est aussi le **point de comparaison** de toute stratégie à venir : faire moins bien que « acheter et dormir » ne mérite pas de complexité.

Trois nouveautés par rapport au lecteur de la Moteur 2, toutes dans `initialize` :

1. **SymbolProperties** — la fiche d'identité de l'instrument : devise, tick de prix, et surtout la **taille de lot**. Le défaut LEAN est un lot de 1 : correct pour une action, absurde pour BTC. On passe le lot à 0.00000001 (1 satoshi) pour autoriser les quantités fractionnaires.
2. **SecurityExchangeHours.always_open(TimeZones.UTC)** — le perp crypto cote 24/7. Ces heures portent aussi le **fuseau de la donnée** (elles remplacent le `TimeZones.UTC` passé à `add_data` en Moteur 2 — le piège du 8 mars reste couvert).
3. **FeeModel custom** — le défaut mesuré en Moteur 2 est le barème Interactive Brokers (min 1 \$/ordre). Pour un perp Binance, le vrai coût taker est **0,04 % du notionnel** (quantité × prix). C'est LA constante de frais de toute la formation.

Le code

Fichier complet : `buyhold.py`. Le lecteur `BtcUsdtHourly` est celui de la Moteur 2, inchangé — principe de la source de vérité : on ne retouche pas ce qui est validé. Les nouveautés :

```
FRAIS_TAKER = 0.0004      # 0,04 % Binance USD$-M — LA constante de frais de la
                             formation
CAPITAL = 100_000
```

```
class FraisTakerBinance(FeeModel):
    """0,04 % du notionnel, comme un ordre marché (taker) sur Binance USD$-M."""

    def get_order_fee(self, parameters):
        notionnel = abs(parameters.order.quantity) * parameters.security.price
        return OrderFee(CashAmount(notionnel * FRAIS_TAKER, "USD"))
```

```
class BuyHold(QCAlgorithm):

    def initialize(self):
        # Bornes adaptatives : lues dans le CSV (jamais de dates figées)
        with open(DATA_FILE) as f:
            rows = f.read().splitlines()
            premier = datetime.strptime(rows[1][:19], "%Y-%m-%d %H:%M:%S")
            dernier = datetime.strptime(rows[-1][:19], "%Y-%m-%d %H:%M:%S")
            self.set_start_date(premier.year, premier.month, premier.day)
            self.set_end_date(dernier.year, dernier.month, dernier.day)
            self.set_cash(CAPITAL)
            self.set_time_zone(TimeZones.UTC)
```

```

# SymbolProperties : le point NOUVEAU de ce backtest.
# lot_size = 1e-8 (1 satoshi) => quantités FRACTIONNAIRES autorisées.
# Sans ça, lot par défaut = 1 : set_holdings arrondirait à 1 BTC entier !
proprietes = SymbolProperties(
    "BTCUSDT perpetuel USDS-M", # description libre
    "USD", # devise de cotation (USDT assimilé à USD – simplification
assumée)
    1, # multiplicateur de contrat
    0.1, # tick de prix minimal (0.1 USDT sur Binance)
    0.00000001, # taille de lot : 1 satoshi
    "BTCUSDT") # ticker marché
# Crypto = 24/7 : bourse toujours ouverte, EN UTC (ces heures portent aussi
# le fuseau de la donnée)
heures = SecurityExchangeHours.always_open(TimeZones.UTC)

securite = self.add_data(BtcUsdtHourly, "BTCUSDT", proprietes, heures,
    Resolution.HOUR)
securite.set_fee_model(FraisTakerBinance())
self.btc = securite.symbol

self.achete = False
self.fill_prix = None
self.frais_totaux = 0.0
self.dernier_close = None

def on_data(self, data: Slice):
    if self.btc not in data:
        return
    self.dernier_close = float(data[self.btc].value)
    if not self.achete:
        self.achete = True
        self.set_holdings(self.btc, 1.0) # tout le capital sur BTC, une seule
fois

def on_order_event(self, event: OrderEvent):
    if event.status == OrderStatus.FILLED:
        self.fill_prix = float(event.fill_price)
        self.frais_totaux += float(event.order_fee.value.amount)
        self.log(f"FILL {event.utc_time:%Y-%m-%d %H:%M} UTC | "
            f"qté={event.fill_quantity} BTC @ {event.fill_price} | "
            f"frais={event.order_fee.value.amount:.2f} $")

def on_end_of_algorithm(self):
    # — SOL DE VÉRITÉ : on recalcule l'équité finale À LA MAIN —
    qte = float(self.portfolio[self.btc].quantity)
    cash = float(self.portfolio.cash)
    equite_lean = float(self.portfolio.total_portfolio_value)
    equite_main = cash + qte * self.dernier_close
    rendement_btc = self.dernier_close / self.fill_prix - 1
    self.log(f"Sol de vérité : qté={qte:.8f} BTC | fill={self.fill_prix} | "
        f"dernier close={self.dernier_close}")
    self.log(f"BTC seul : {rendement_btc:+.4%} | frais
payés={self.frais_totaux:.2f} $")
    self.log(f"Équité LEAN={equite_lean:.2f} vs main={equite_main:.2f} | "
        f"écart={equite_lean - equite_main:+.6f} $")

```

Un détail d'API : `add_data(type, ticker, proprietes, heures, resolution)` est une surcharge spécifique (Algorithm/QCAlgorithm.Python.cs) qui enregistre propriétés et heures **avant** de créer la souscription.

Exécution et résultat mesuré

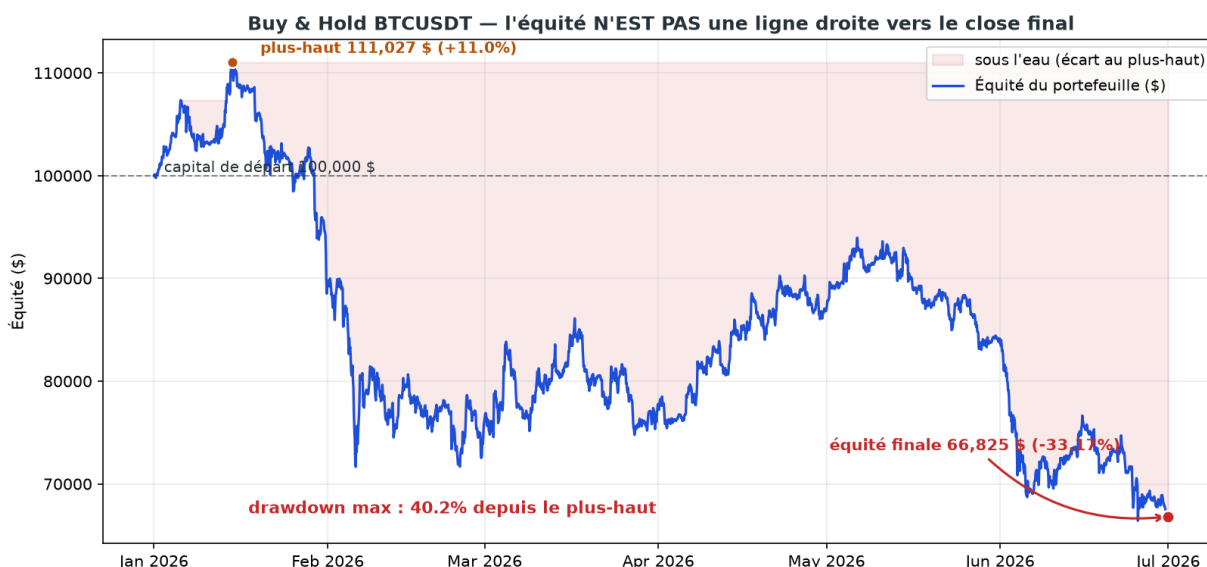
```
cd lean/Launcher/bin/Release
dotnet QuantConnect.Lean.Launcher.dll `
  --algorithm-language Python `
  --algorithm-type-name BuyHold `
  --algorithm-location "../../../algorithms/buyhold.py"
```

Sortie mesurée sur notre historique (janvier→juin 2026 — tes chiffres évolueront avec le CSV, la structure doit être identique) :

```
FILL 2026-01-01 01:00 UTC | qté=1.13599564 BTC @ 87773.4 | frais=39.88 $
Sol de vérité : qté=1.13599564 BTC | fill=87773.4 | dernier close=58605.4
BTC seul : -33.2310% | frais payés=39.88 $
Équité LEAN=66825.40 vs main=66825.40 | écart=+0.000000 $
```

```
STATISTICS:: End Equity 66825.40
STATISTICS:: Net Profit -33.175%
STATISTICS:: Drawdown 40.200%
STATISTICS:: Total Fees $39.88
```

Trois observations : le fill a lieu **au close de la première barre** (marché 24/7 = toujours ouvert, la règle générale de la Moteur 2 s'applique toujours, jamais de MarketOnOpen) ; la quantité est **fractionnaire** (le lot d'un satoshi fonctionne) ; les frais font bien 0,04 % du notionnel.



Courbe d'équité du Buy & Hold BTCUSDT : départ 100 000 \$, plus-haut en janvier, longue glissade avec un drawdown maximal de 40 % depuis le plus-haut.

Figure — Le rendement final ne dit rien du chemin : drawdown de 40 % depuis le plus-haut. (Script : `fig_buyhold_equite.py`)

Le sol de vérité, ligne à ligne

écart=0 prouve la cohérence *interne*. Reste la cohérence *externe* : chaque nombre doit se déduire **du CSV et des constantes**, sans faire confiance au moteur :

```

capital = 100_000
fill     = 87_773.4          # close de la barre 1 (2e ligne du CSV)
qte      = 1.13599564        # la quantité du FILL
close_f  = 58_605.4          # close de la DERNIÈRE ligne du CSV

notionnel = qte * fill                # 99 710.20 $ (ce qu'on a payé en
BTC)
frais     = notionnel * 0.0004        # 39.88 $ (0,04 % du notionnel)
cash      = capital - notionnel - frais # 249.92 $ (ce qui reste en
caisse)
equite    = cash + qte * close_f      # 66 825.40 $ (= End Equity de LEAN
!)
```

Et la quantité, d'où sort-elle ? `set_holdings(1.0)` ne divise pas 100 000 par le prix : LEAN vise le portefeuille **moins un tampon** de 0,25 % (`FreePortfolioValuePercentage = 0.0025` dans `Common/AlgorithmSettings.cs` — pour que l'ordre ne soit pas rejeté si le prix bouge), retranche les **frais estimés** (notre modèle est consulté avant l'exécution), puis arrondit au **lot**. Résultat : **~99,7 % investi, jamais 100 %** — comportement par défaut, documenté et réglable, pas un bug. (Le calcul exact est itératif : `Common/Securities/BuyingPowerModel.cs`.)

On peut même décomposer l'écart entre le Buy & Hold « théorique » (`close_fin / open_début - 1` = -33,106 %) et le Net Profit de LEAN :

```

(a) entrée au close de la barre 1, pas à l'open : -33,231 % (-0,125 pt)
(b) × fraction réellement investie (0,99710) : -33,135 % (+0,096 pt)
(c) - frais/capital (0,040 %) : -33,175 % (= Net Profit LEAN ✓)
```

Même Buy & Hold a une **sémantique d'exécution** : trois choix d'implémentation (quand on entre, combien on investit, ce qu'on paie) font 0,07 point d'écart. Sur des stratégies qui tradent des centaines de fois, ces choix **dominent** le résultat — c'est tout l'enjeu de la future parité entre moteurs.

Falsification : sans `SymbolProperties`

On affirme que sans lot d'un satoshi, LEAN arrondit à 1 BTC entier. Preuve — `buyhold_sanslot.py` est la copie conforme, avec l'`add_data` court de la Moteur 2 (sans propriétés). Résultat mesuré :

```

FILL 2026-01-01 01:00 UTC | qté=1.0 BTC @ 87773.4 | frais=35.11 $
STATISTICS:: Net Profit -29.203%
```

`set_holdings(1.0)` voulait ~1,136 BTC ; le lot par défaut a arrondi à **1 BTC tout rond** : ~12 % du capital est resté en cash, le backtest a silencieusement testé « investir 88 % ». Le plus vicieux : le résultat est **meilleur** (-29,2 % vs -33,2 % — moins investi = moins perdu en marché baissier), aucune erreur, aucun warning, et le sol de vérité *interne* affiche toujours écart=0. Seule la comparaison au **calcul externe** révèle qu'on n'a pas backtesté la stratégie qu'on croyait. Avec un capital inférieur au prix du BTC, c'est pire : quantité arrondie à **zéro**, aucun ordre passé.

Ce qu'il faut retenir

- **Sol de vérité** : le backtest le plus simple doit se recalculer à la main au centime près — cohérence interne (écart=0) ET externe (bilan depuis le CSV). C'est le premier test qu'on rejouera sur `vectorbt`.
- **`SymbolProperties` d'abord** (lot 1 satoshi, bourse 24/7 UTC) et **frais explicites** (0,04 % du notionnel) : les deux défauts de LEAN — lot 1, barème IB — faussent le backtest **en silence**.
- `set_holdings(1.0)` investit ~99,7 % (tampon 0,25 % + frais estimés + arrondi au lot) ; ne « corrige » pas en visant 1.05, tu créerais du levier involontaire.

- Le rendement final ne montre pas le **chemin** : drawdown max ~40 % ici — la métrique qu'on outillera en Stratégie 5.

Précédent : [Moteur 2 — Le moteur et nos données](#) · Suite : [Stratégie 2 — Suivre la tendance : SMA et MACD](#), les premières vraies stratégies et le premier face-à-face avec ce Buy & Hold.